Data Structures

Mushtari Sadia

Hashing and HashTable

Need for Hash data structure

Every day, the data on the internet is increasing multifold and it is always a struggle to store this data efficiently. In day-to-day programming, this amount of data might not be that big, but still, it needs to be stored, accessed, and processed easily and efficiently. A very common data structure that is used for such a purpose is the Array data structure.

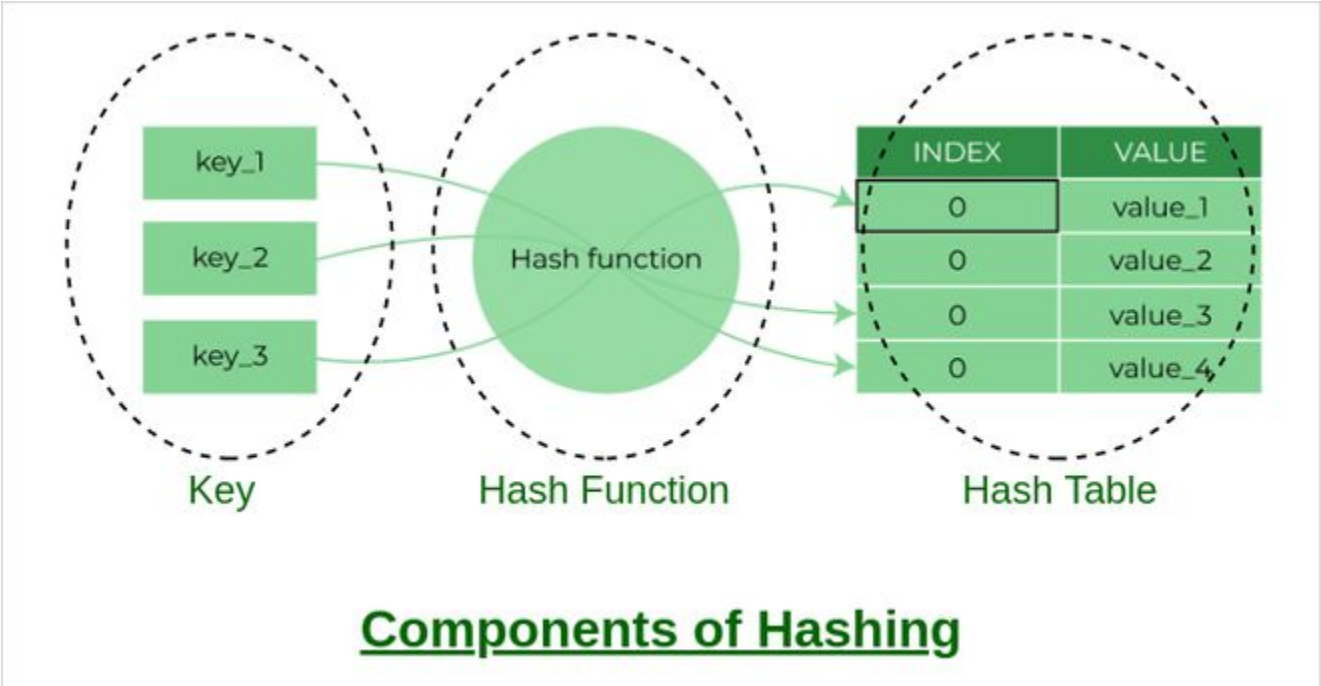
Now the question arises if Array was already there, what was the need for a new data structure! The answer to this is in the word “efficiency“. Though storing in Array takes O(1) time, searching in it takes at least O(log n) time. This time appears to be small, but for a large data set, it can cause a lot of problems and this, in turn, makes the Array data structure inefficient.

So now we are looking for a data structure that can store the data and search in it in constant time, i.e. in O(1) time. This is how Hashing data structure came into play. With the introduction of the Hash data structure, it is now possible to easily store data in constant time and retrieve them in constant time as well.

Components of Hashing

There are majorly three components of hashing:

- **Key**: A Key can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
- **Hash Function**: The hash function receives the input key and returns the index of an element in an array called a hash table. The index is known as the hash index.
- **Hash Table**: Hash table is a data structure that maps keys to values using a special function called a hash function. Hash stores the data in an associative manner in an array where each data value has its own unique index.


key_1
key_2
key_3
Hash function
INDEX
VALUE
0
value_1
0
value_2
0
value_3
0
value_4
Key
Hash Function
Hash Table
Components of Hashing

How does Hashing work?

Suppose we have a set of strings {“ab”, “cd”, “efg”} and we would like to store it in a table.

Our main objective here is to search or update the values stored in the table quickly in O(1) time and we are not concerned about the ordering of strings in the table. So the given set of strings can act as a key and the string itself will act as the value of the string, but how to store the value corresponding to the key?

How does Hashing work?

- **Step 1:** We know that hash functions (which is some mathematical formula) are used to calculate the hash value which acts as the index of the data structure where the value will be stored.
- **Step 2:** So, let's assign
    - "a" = 1,
    - "b"=2, .. etc, to all alphabetical characters.
- **Step 3:** Therefore, the numerical value by summation of all characters of the string:
    - *"ab" = 1 + 2 = 3*,
    - *"cd" = 3 + 4 = 7* ,
    - *"efg" = 5 + 6 + 7 = 18*
- **Step 4:** Now, assume that we have a table of size 7 to store these strings. The hash function that is used here is the sum of the characters in **key mod Table size**. We can compute the location of the string in the array by taking the **sum(string) mod 7**.
- **Step 5:** So we will then store
    - "ab" in 3 mod 7 = 3,
    - "cd" in 7 mod 7 = 0, and
    - "efg" in 18 mod 7 = 4.

How does Hashing work?

- “ab” in 3 mod 7 = 3,
- “cd” in 7 mod 7 = 0, and
- “efg” in 18 mod 7 = 4.

0	1	2	3	4	5	6
cd			ab	efg		

Problem with Hashing

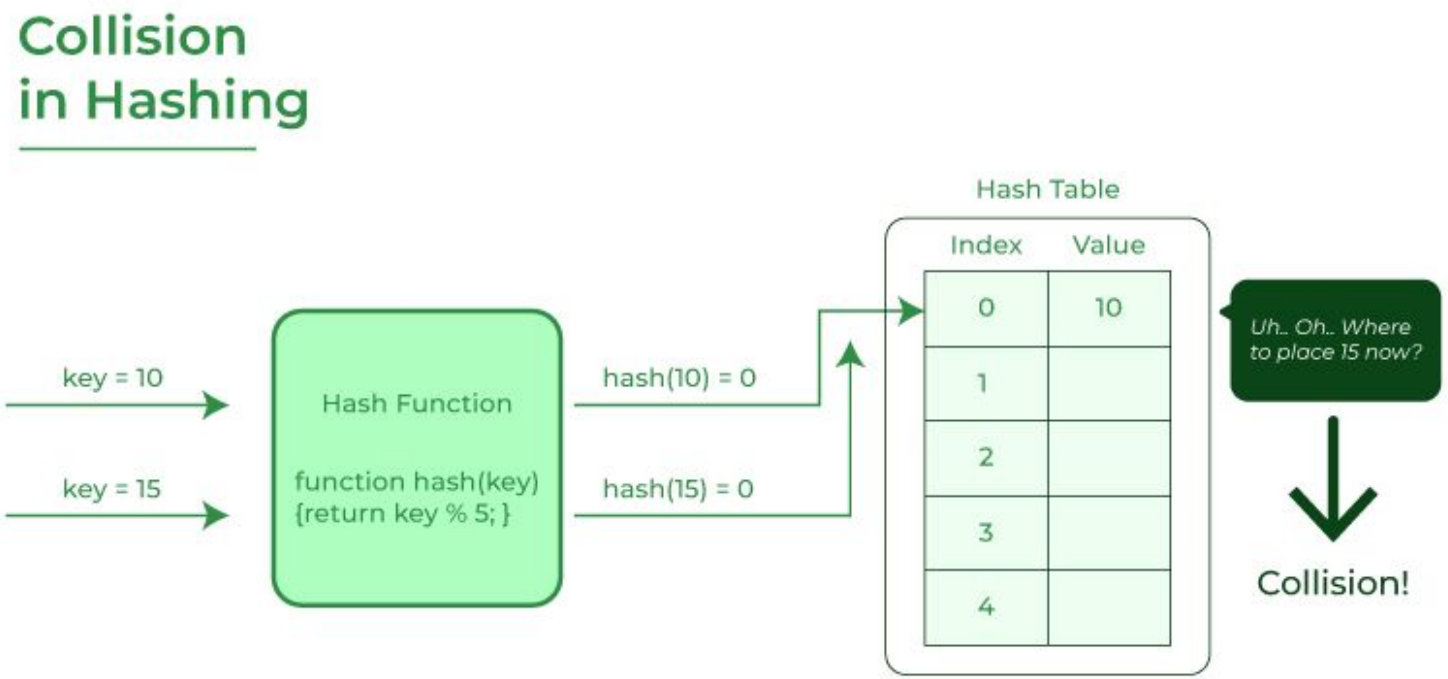
If we consider the above example, the hash function we used is the sum of the letters, but if we examined the hash function closely then the problem can be easily visualized that for different strings same hash value is begin generated by the hash function.

For example: {“ab”, “ba”} both have the same hash value, and string {“cd”,”be”} also generate the same hash value, etc. This is known as **collision** and it creates problem in searching, insertion, deletion, and updating of value

What is collision?

The hashing process generates a small number for a big key, so there is a possibility that two keys could produce the same value. The situation where the newly inserted key maps to an already occupied, and it must be handled using some collision handling technology.

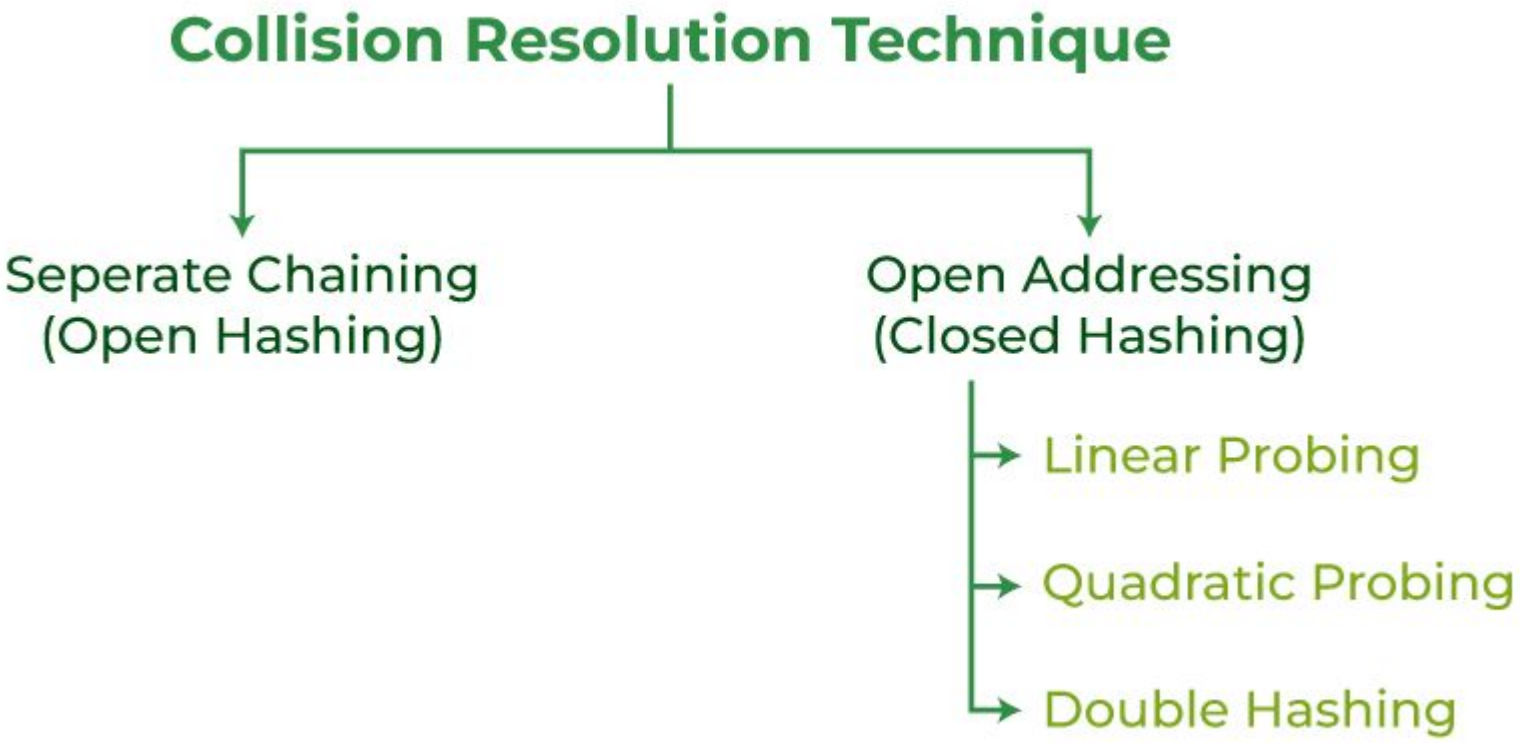
What is collision?



How to handle Collisions?

There are mainly two methods to handle collision:

1. Separate Chaining:
2. Open Addressing:


Collision Resolution Technique
Seperate Chaining
(Open Hashing)
Open Addressing
(Closed Hashing)
Linear Probing
Quadratic Probing
Double Hashing

Separate Chaining

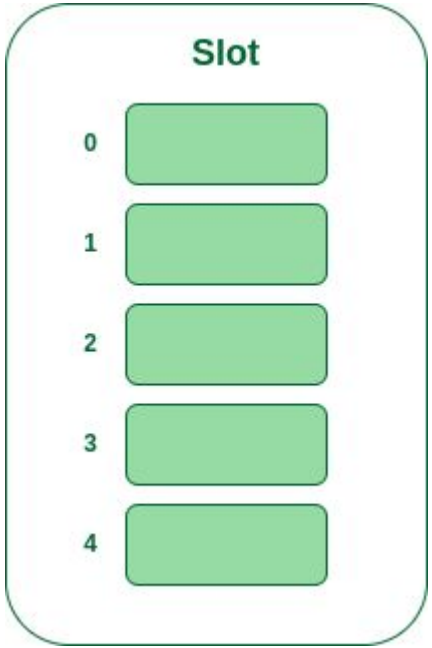
The idea is to make each cell of the hash table point to a linked list of records that have the same hash function value. Chaining is simple but requires additional memory outside the table.

Example: We have given a hash function and we have to insert some elements in the hash table using a separate chaining method for collision resolution technique.

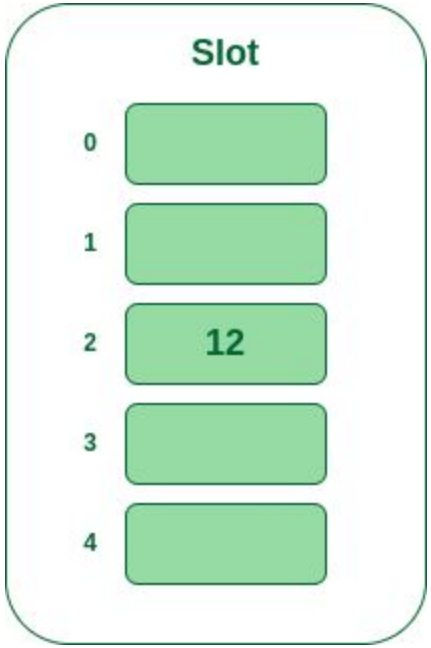
```
Hash function = key % 5,

Elements = 12, 15, 22, 25 and 37.
```

Let’s see step by step approach to how to solve the above problem:

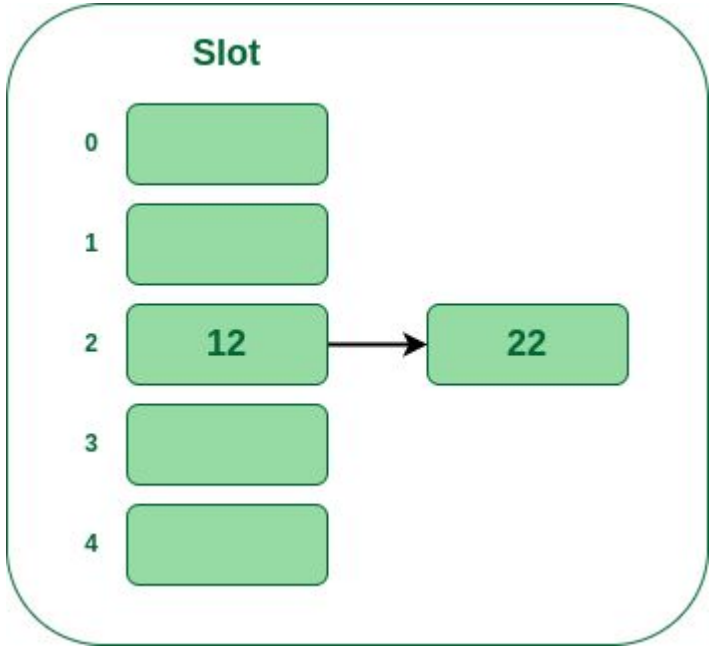
- **Step 1:** First draw the empty hash table which will have a possible range of hash values from 0 to 4 according to the hash function provided.



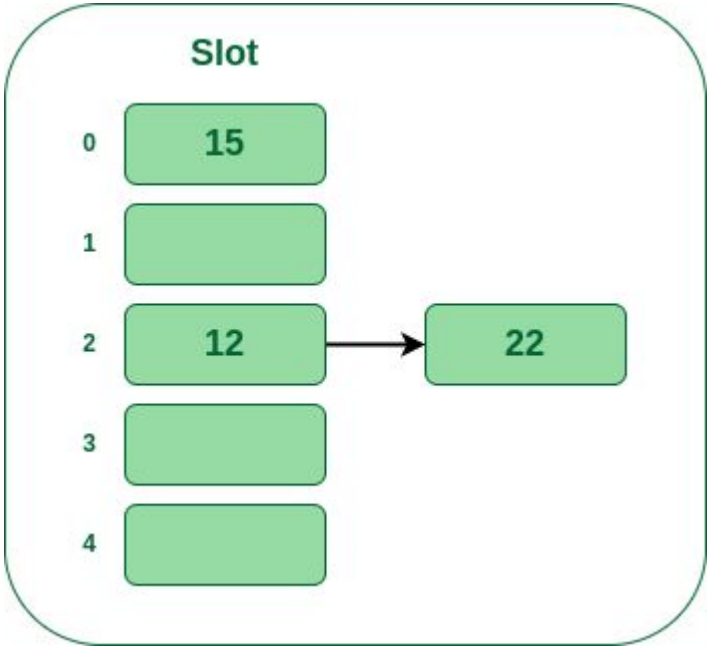
- **Step 2:** Now insert all the keys in the hash table one by one. The first key to be inserted is 12 which is mapped to bucket number 2 which is calculated by using the hash function 12%5=2.



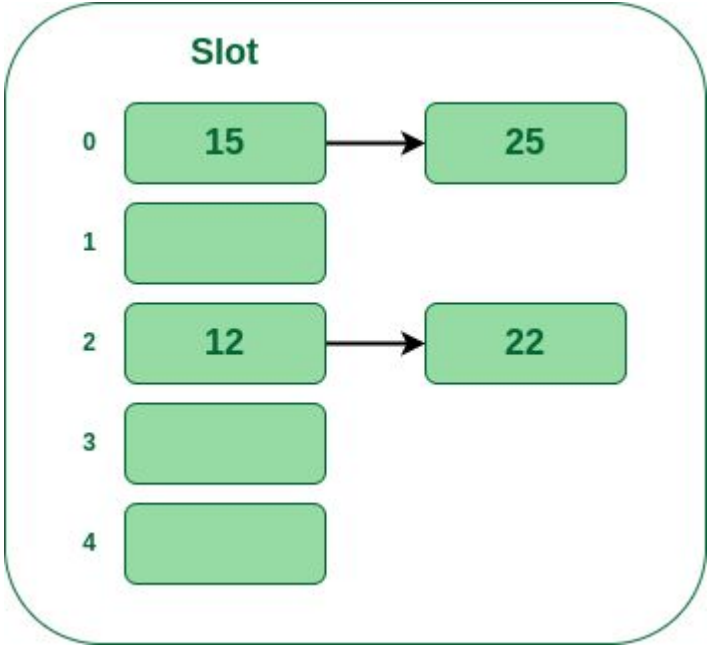
- **Step 3:** Now the next key is 22. It will map to bucket number 2 because 22%5=2. But bucket 2 is already occupied by key 12.



- **Step 4:** The next key is 15. It will map to slot number 0 because 15%5=0.



- **Step 5:** Now the next key is 25. Its bucket number will be 25%5=0. But bucket 0 is already occupied by key 25. So separate chaining method will again handle the collision by creating a linked list to bucket 0.



Separate Chaining

```
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

class SeparateChainingHashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * self.size

    def _hash(self, key):
        return sum(ord(c) for c in key) % self.size
```

```
def insert(self, key, value):
        index = self._hash(key)
        new_node = Node(key, value)
        if self.table[index] is None:
            self.table[index] = new_node
        else:
            current = self.table[index]
            while current.next:
                current = current.next
            current.next = new_node

    def get(self, key):
        index = self._hash(key)
        current = self.table[index]
        while current:
            if current.key == key:
                return current.value
            current = current.next
        return None
```

Separate Chaining

Example usage of Separate Chaining Hash Table:

```
hash_table = SeparateChainingHashTable(10)
hash_table.insert("apple", 5)
hash_table.insert("banana", 7)
hash_table.insert("orange", 2)

print(hash_table.get("apple"))   # Output: 5
print(hash_table.get("banana"))  # Output: 7
print(hash_table.get("orange"))  # Output: 2
print(hash_table.get("grape"))   # Output: None (Key not found)
```

Open Addressing

In open addressing, all elements are stored in the hash table itself. Each table entry contains either a record or NIL. When searching for an element, we examine the table slots one by one until the desired element is found or it is clear that the element is not in the table.

Linear Probing

In linear probing, the hash table is searched sequentially that starts from the original location of the hash. If in case the location that we get is already occupied, then we check for the next location.

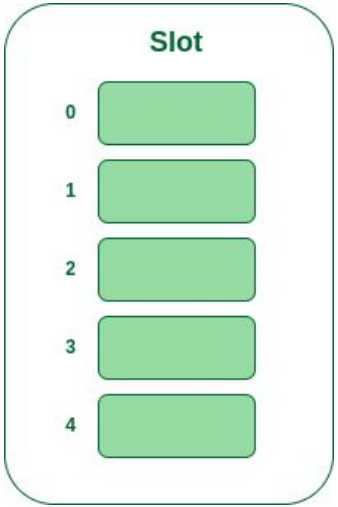
Algorithm:

1. *Calculate the hash key. i.e.* ***key = data % size***
2. *Check, if* ***hashTable[key]*** *is empty*
    1. *store the value directly by* ***hashTable[key] = data***
3. *If the hash index already has some value then*
    1. *check for next index using* ***key = (key+1) % size***
4. *Check, if the next index is available hashTable[key] then store the value. Otherwise try for next index.*
5. *Do the above process till we find the space.*

Linear Probing

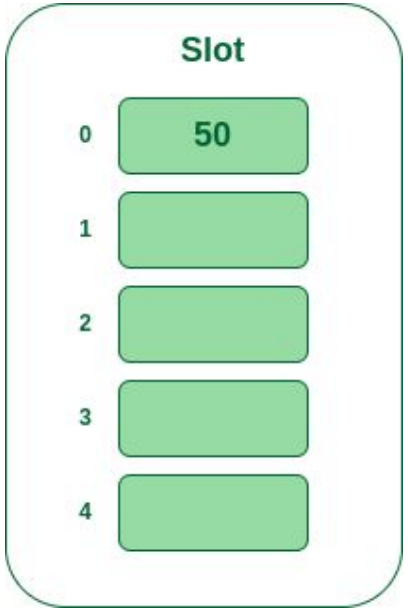
Example: Let us consider a simple hash function as “key mod 5” and a sequence of keys that are to be inserted are 50, 70, 76, 85, 93.

- **Step 1:** First draw the empty hash table which will have a possible range of hash values from 0 to 4 according to the hash function provided.



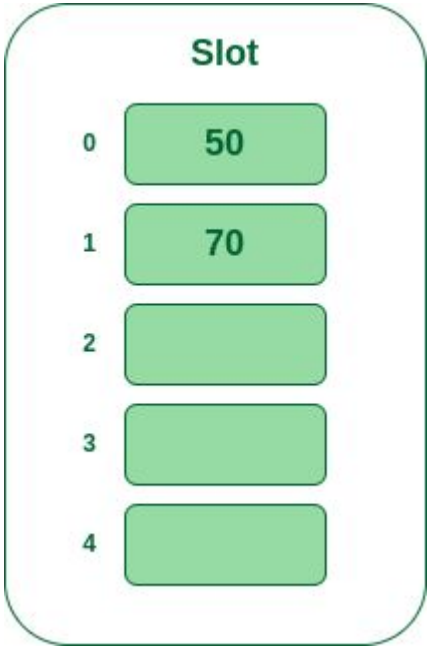
Linear Probing

- **Step 2:** Now insert all the keys in the hash table one by one. The first key is 50. It will map to slot number 0 because 50%5=0. So insert it into slot number 0.



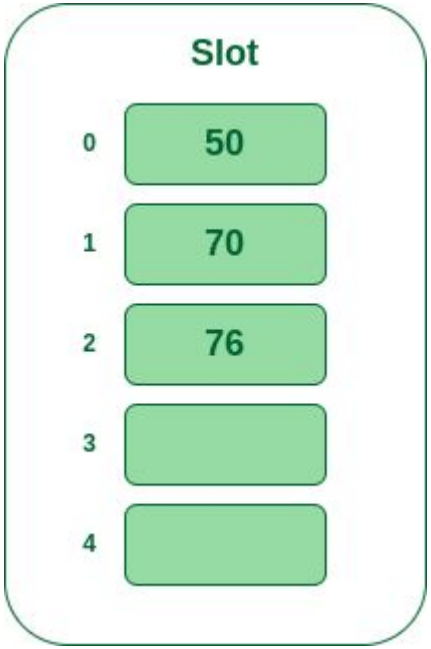
Linear Probing

- **Step 3:** The next key is 70. It will map to slot number 0 because 70%5=0 but 50 is already at slot number 0 so, search for the next empty slot and insert it.



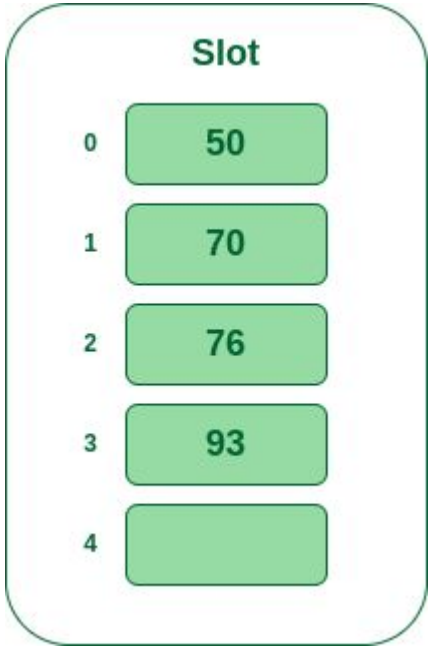
Linear Probing

- **Step 4:** The next key is 76. It will map to slot number 1 because 76%5=1 but 70 is already at slot number 1 so, search for the next empty slot and insert it.



Linear Probing

- **Step 5:** The next key is 93 It will map to slot number 3 because 93%5=3, So insert it into slot number 3.



Quadratic Probing

Quadratic probing is an open addressing scheme in computer programming for resolving hash collisions in hash tables. Quadratic probing operates by taking the original hash index and adding successive values of an arbitrary quadratic polynomial until an open slot is found.

An example sequence using quadratic probing is:

$$H + 1^2, H + 2^2, H + 3^2, H + 4^2 \ldots\ldots\ldots\ldots\ldots H + k^2$$

This method is also known as the mid-square method because in this method we look for i^2'th probe (slot) in i'th iteration and the value of i = 0, 1, . . . n – 1. We always start from the original hash location. If only the location is occupied then we check the other slots.

Quadratic Probing

Let hash(x) be the slot index computed using the hash function and n be the size of the hash table.

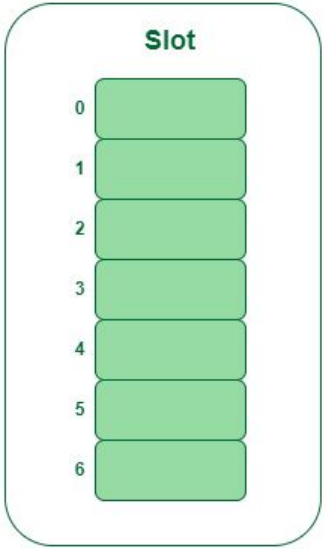
If the slot hash(x) % n is full, then we try (hash(x) + 1^2) % n.
If (hash(x) + 1^2) % n is also full, then we try (hash(x) + 2^2) % n.
If (hash(x) + 2^2) % n is also full, then we try (hash(x) + 3^2) % n.
*This process will be repeated for all the values of **i** until an empty slot is found*

Example: Let us consider table Size = 7, hash function as Hash(x) = x % 7 and collision resolution strategy to be f(i) = i^2 . Insert = 22, 30, and 50

Quadratic Probing

Example: Let us consider table Size = 7, hash function as Hash(x) = x % 7 and collision resolution strategy to be $f(i) = i^2$. Insert = 22, 30, and 50

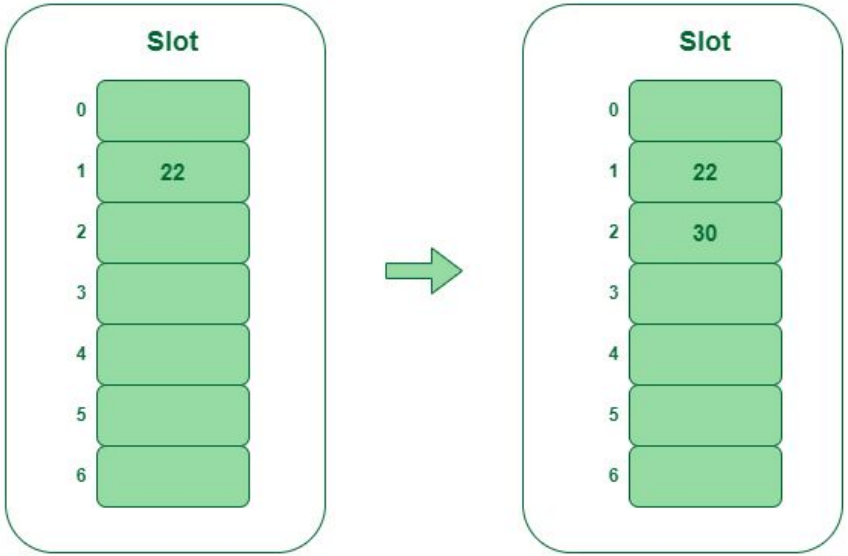
- **Step 1:** Create a table of size 7.



Hash table

Quadratic Probing

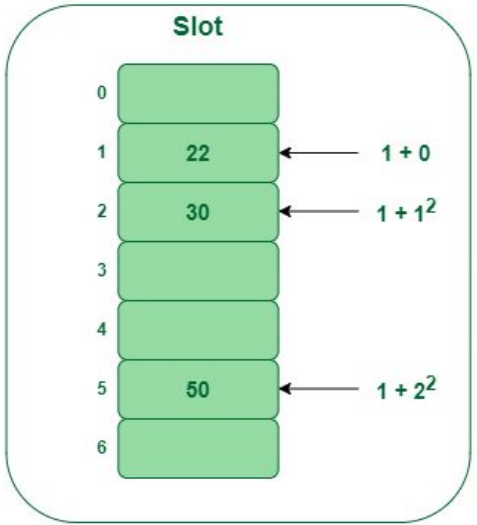
- **Step 2** – Insert 22 and 30
  - Hash(25) = 22 % 7 = 1, Since the cell at index 1 is empty, we can easily insert 22 at slot 1.
  - Hash(30) = 30 % 7 = 2, Since the cell at index 2 is empty, we can easily insert 30 at slot 2.



Insert key 22 and 30 in the hash table

Quadratic Probing

- **Step 3:** Inserting 50
  - Hash(25) = 50 % 7 = 1
  - In our hash table slot 1 is already occupied. So, we will search for slot $1+1^2$, i.e. 1+1 = 2,
  - Again slot 2 is found occupied, so we will search for cell $1+2^2$, i.e.1+4 = 5,
  - Now, cell 5 is not occupied so we will place 50 in slot 5.



Insert key 50 in the hash table

Double Hashing

Double hashing is a collision resolving technique in Open Addressed Hash tables. Double hashing make use of two hash function,

- The first hash function is **h1(k)** which takes the key and gives out a location on the hash table. But if the new location is not occupied or empty then we can easily place our key.
- But in case the location is occupied (collision) we will use secondary hash-function **h2(k)** in combination with the first hash-function **h1(k)** to find the new location on the hash table.

This combination of hash functions is of the form

```
h(k, i) = (h1(k) + i * h2(k)) % n
```

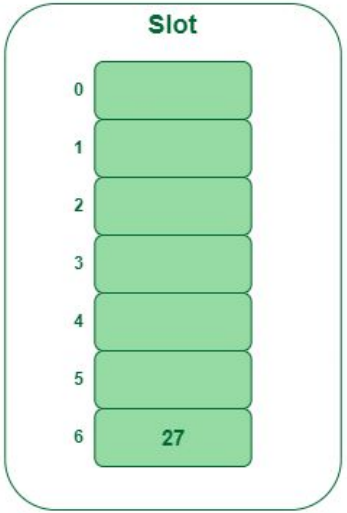
where

- i is a non-negative integer that indicates a collision number,
- k = element/key which is being hashed
- n = hash table size.

Double Hashing

Example: Insert the keys 27, 43, 692, 72 into the Hash Table of size 7. where first hash-function is **h1(k) = k mod 7** and second hash-function is **h2(k) = 1 + (k mod 5)**

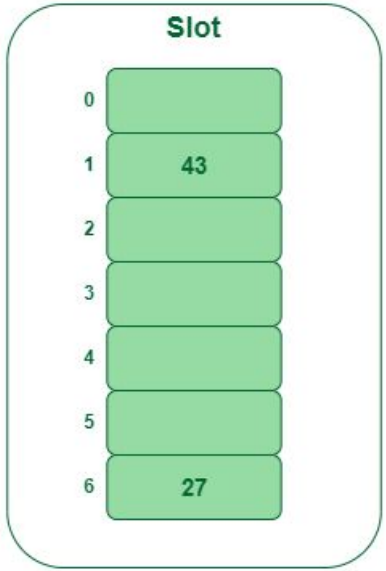
- **Step 1:** Insert 27
  - 27 % 7 = 6, location 6 is empty so insert 27 into 6 slot.



Insert key 27 in the hash table

Double Hashing

- **Step 2:** Insert 43
  - 43 % 7 = 1, location 1 is empty so insert 43 into 1 slot.



Insert key 43 in the hash table

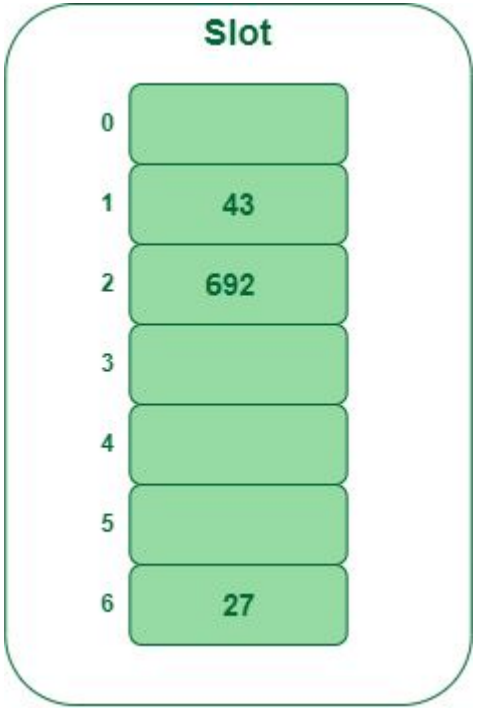
Double Hashing

- **Step 3:** Insert 692
  - 692 % 7 = 6, but location 6 is already being occupied and this is a collision
  - So we need to resolve this collision using double hashing.

h_{new} = [h1(692) + i * (h2(692)] % 7
= [6 + 1 * (1 + 692 % 5)] % 7
= 9 % 7
= 2

Now, as 2 is an empty slot,
so we can insert 692 into 2nd slot.

Double Hashing



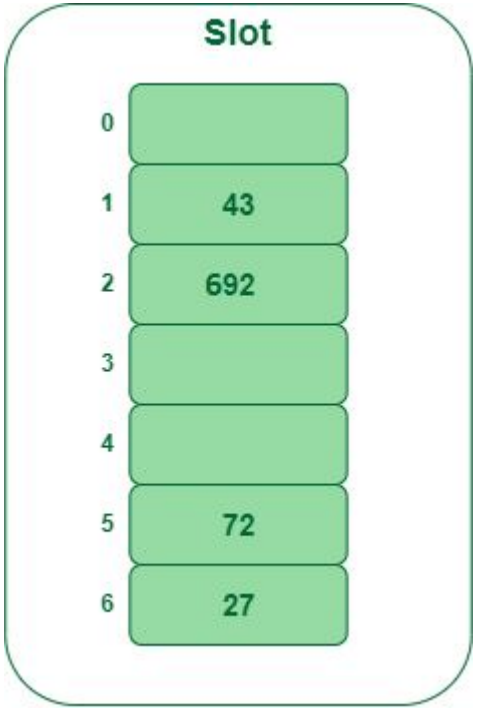
Double Hashing

- **Step 4:** Insert 72
  - 72 % 7 = 2, but location 2 is already being occupied and this is a collision.
  - So we need to resolve this collision using double hashing.

h_{new} = [h1(72) + i * (h2(72)] % 7
= [2 + 1 * (1 + 72 % 5)] % 7
= 5 % 7
= 5,

Now, as 5 is an empty slot,
so we can insert 72 into 5th slot.

Double Hashing



Double Hashing

```
class OpenAddressingHashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * self.size

    def _hash1(self, key):
        return sum(ord(c) for c in key) % self.size

    def _hash2(self, key):
        # Using a simple secondary hash function to avoid collisions with size as a factor.
        # This is just an example and may not be ideal in practice.
        return 1 + (sum(ord(c) for c in key) % (self.size - 1))

    def _probe(self, key, attempt):
        return (self._hash1(key) + attempt * self._hash2(key)) % self.size
```

Double Hashing

```
def insert(self, key, value):
    attempt = 0
    index = self._hash1(key)
    while self.table[index] is not None:
        attempt += 1
        index = self._probe(key, attempt)
    self.table[index] = (key, value)

def get(self, key):
    attempt = 0
    index = self._hash1(key)
    while self.table[index] is not None:
        if self.table[index][0] == key:
            return self.table[index][1]
        attempt += 1
        index = self._probe(key, attempt)
    return None
```

Double Hashing

Example usage of Open Addressing Hash Table with Double Hashing:

```
hash_table = OpenAddressingHashTable(10)
hash_table.insert("apple", 5)
hash_table.insert("banana", 7)
hash_table.insert("orange", 2)

print(hash_table.get("apple"))   # Output: 5
print(hash_table.get("banana"))  # Output: 7
print(hash_table.get("orange"))  # Output: 2
print(hash_table.get("grape"))   # Output: None (Key not found)
```

Key-Indexed Searching - Setup

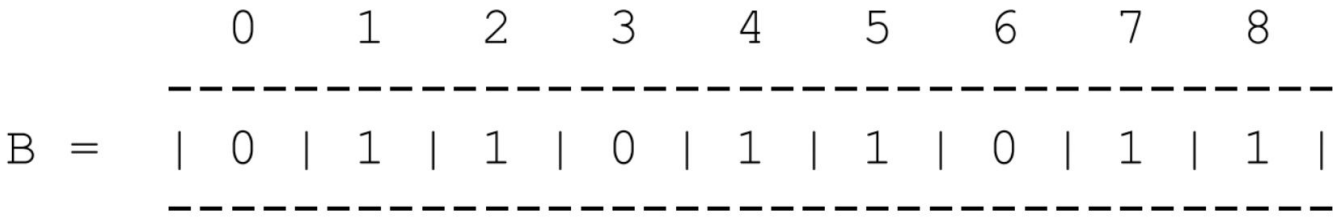
```
      0   1   2   3   4   5
    ---------------------------
A = | 5 | 1 | 7 | 2 | 8 | 4 |
    ---------------------------
```

```
      0   1   2   3   4   5   6   7   8
    -------------------------------------
B = | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
    -------------------------------------
```

```
      0   1   2   3   4   5   6   7   8
    -------------------------------------
B = | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
    -------------------------------------
```

(Slides adapted from Prantik Paul)

Key-Indexed Searching



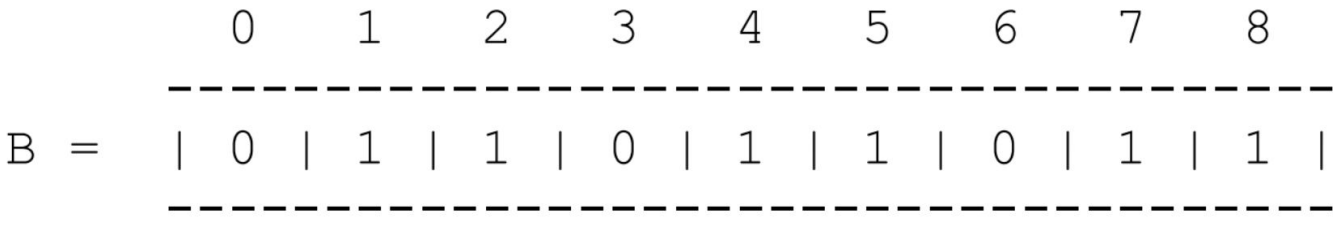
Search 8

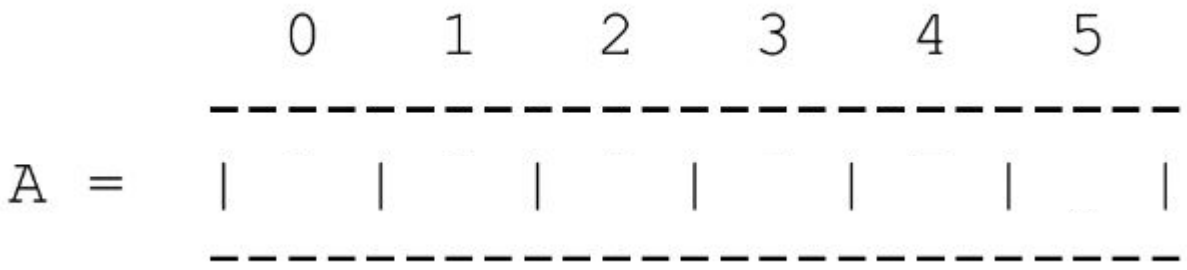
B[8] = 1

Search 3

B[3] = 0

Key-Indexed Sorting





Key-Indexing - Restrictions?

Must be Non-Negative Integers

Cannot have Repeated Elements

Out of Place - must account for Space

Key-Indexing - Negative Integers

A =

0	1	2	3	4
3	-3	1	5	-2

Shift Amount Extra!!

B` =

-3	-2	-1	0	1	2	3	4	5
1	1	0	0	1	0	1	0	1

B =

0	1	2	3	4	5	6	7	8
1	1	0	0	1	0	1	0	1

Key-Indexing - Non Distinct Integers

A =

0	1	2	3	4
1	-3	1	5	1

B` =

0	1	2	3	4	5	6	7	8
1	0	0	0	1	0	0	0	1

B =

0	1	2	3	4	5	6	7	8
1	0	0	0	3	0	0	0	1

References

- Introduction to Hashing - Data Structure and Algorithm Tutorials - GeeksforGeeks
- CLRS Book: Chapter 11, page 253